

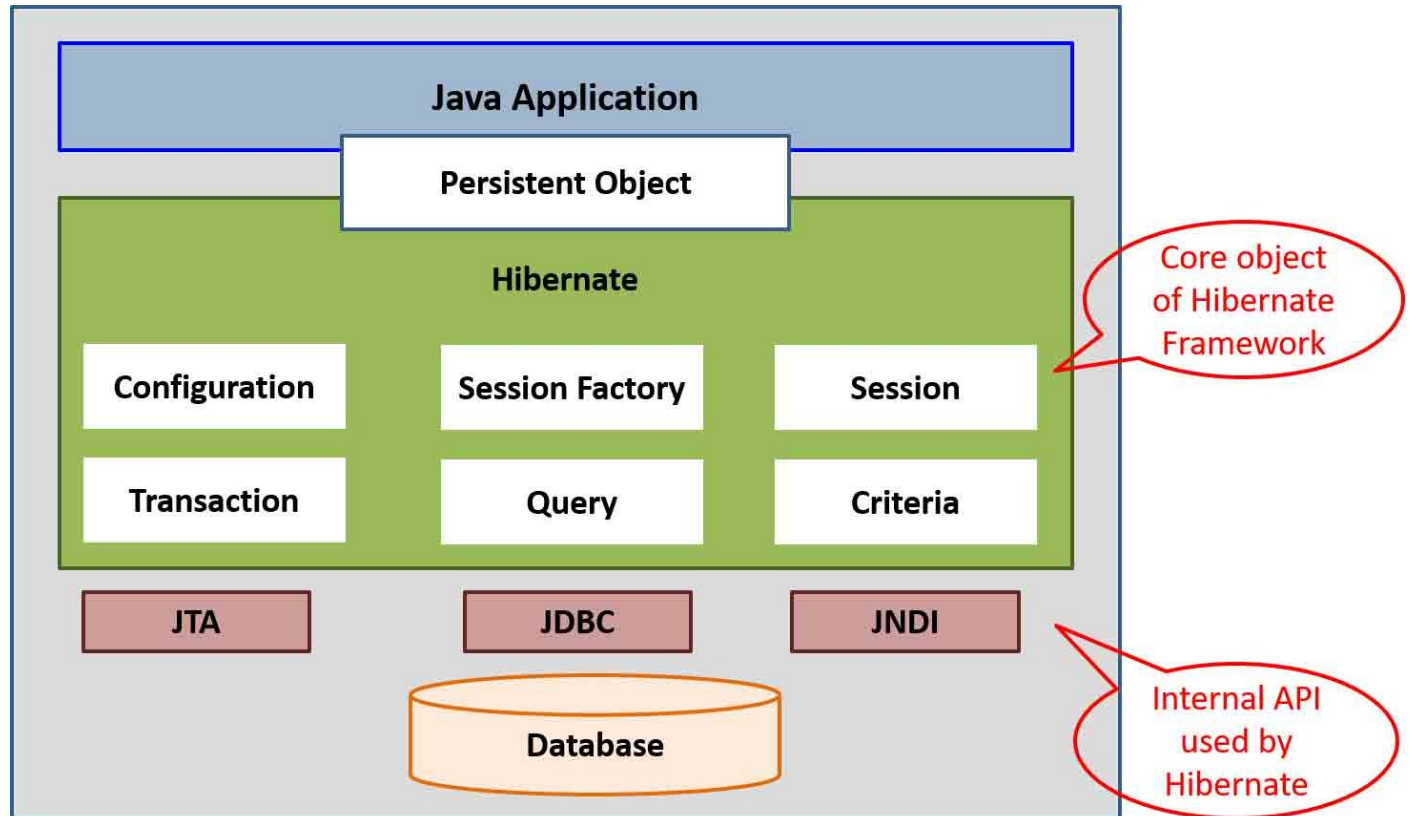
Hibernate

What is Hibernate?

Hibernate is Java framework that help to interact with database.

It is ORM tool. And also Hibernate is like implementation of JPA.

Is use to perform persistence operation on Database with less or no database queries.

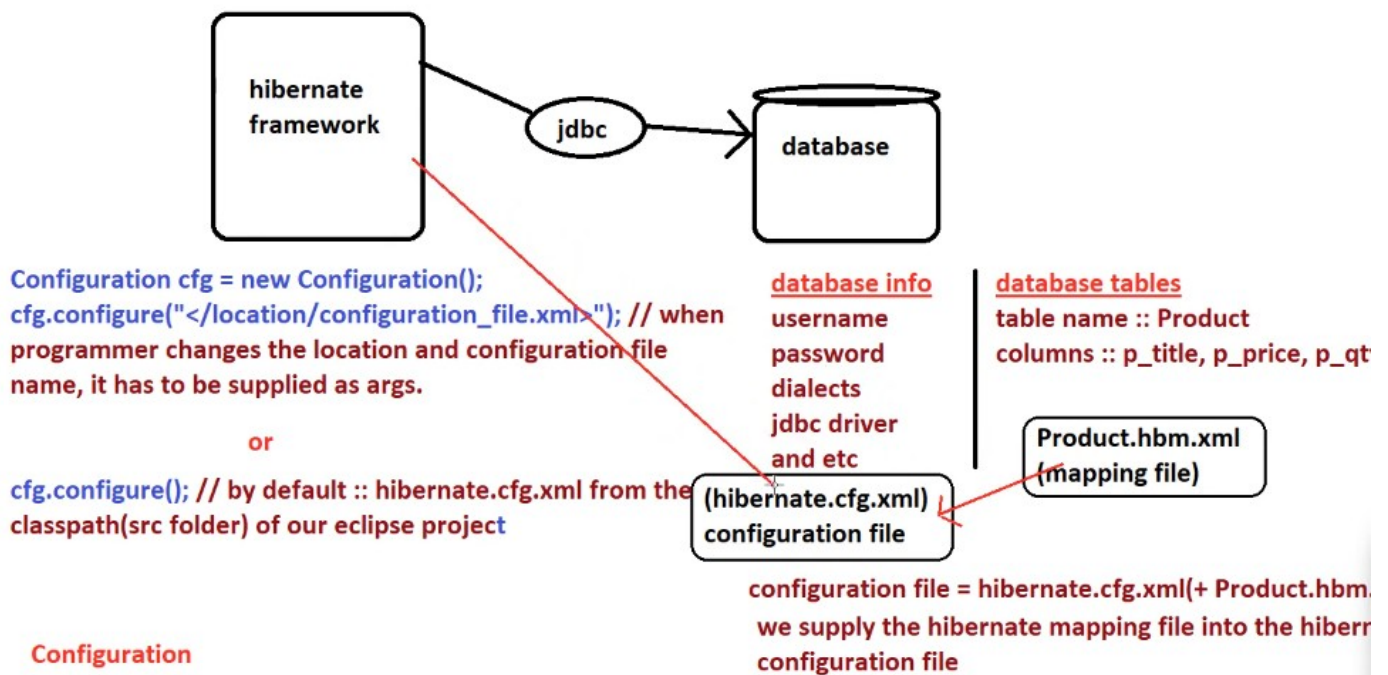
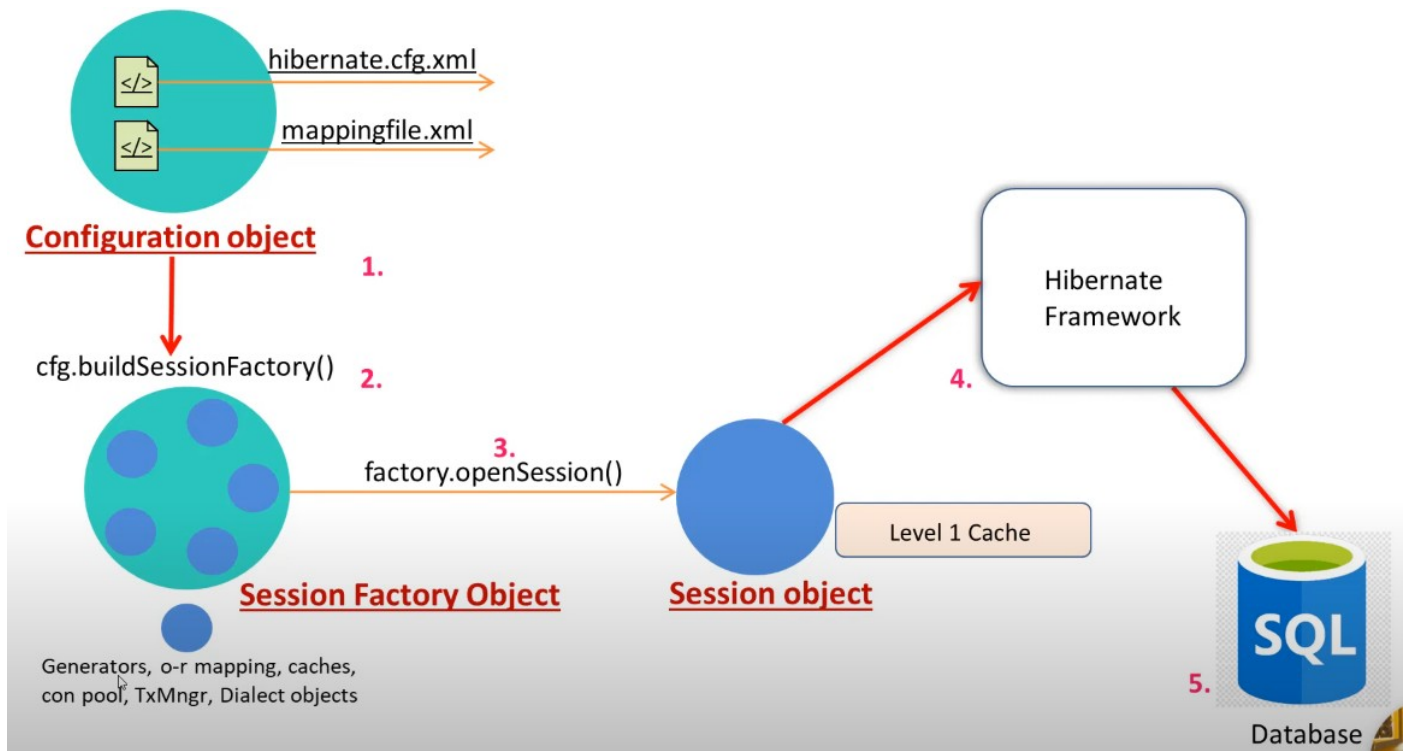


What is DAO Data Access Object Class?

DAO is related to database which we will use for modification or updating in database and make them reusable.

3 important Objects of Hibernate Persistence Logic.

1. Configuration Object – class
2. Session Factory Object – Interface – org.hibernate
3. Session Object



Create Hibernate Simple Project.

1. Create Maven quick start project
2. Add hibernate dependency in pom.xml file

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>7.0.0.Alpha2</version>
  <type>pom</type>
</dependency>
```

3. Create hibernate.cfg.xml file in src/main/java.
4. Search hibernate dtd on google and copy this
5. Write hibernate configuration for JDBC connection in this file.

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>

        <property name="connection.driver_class">com.mysql.jdbc.Driver</property>
        <property
name="connection.url">jdbc:mysql://localhost:3306/hibernate</property>
        <property name="connection.username">root</property>
        <property name="connection.password">root</property>
        <property name="dialect">org.hibernate.dialect.MySQL5Dialect</property>
        <property name="hbm2ddl.auto">update</property>
        <property name="show_sql">true</property>

    </session-factory>
</hibernate-configuration>

```

6. Now in main class file we have to create SessionFactory interface variable.

```

public class App
{
    public static void main( String[] args )
    {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory factory = cfg.buildSessionFactory();
        factory.close();
    }
}

```

```

1 <?xml version='1.0' encoding='utf-8'?>
2 <!DOCTYPE hibernate-configuration SYSTEM
3 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
4
5 <hibernate-configuration>
6     <session-factory>
7         <property name="connection.driver_class">com.mysql.jdbc.Driver</property>
8         <property name="connection.url">jdbc:mysql://localhost:3306/myhiber</property>
9         <property name="connection.username">root</property>
10        <property name="connection.password">root</property>
11        <property name="dialect">org.hibernate.dialect.MySQLDialect</property>
12        <property name="hbm2ddl.auto">update</property>
13        <property name="show_sql">true</property>
14    </session-factory>
15 </hibernate-configuration>
<hibernate-configuration>
    <session-factory>
        <property name="connection.driver_class">com.mysql.jdbc.Driver</property>
        <property name="connection.url">jdbc:mysql://localhost:3306/myhiber</property>
        <property name="connection.username">root</property>
        <property name="connection.password">root</property>
        <property name="dialect">org.hibernate.dialect.MySQL5Dialect</property>
        <property name="hbm2ddl.auto">create</property>
        <property name="show_sql">true</property>
        <mapping class="com.tut.Student" />
        <mapping class="com.tut.Address" />
        <mapping class="com.map.Question" />
        <mapping class="com.map.Answer" />
    </session-factory>
</hibernate-configuration>

```

```

public class App {
    public static void main(String[] args) {
        System.out.println("Project started..");

        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory factory = cfg.buildSessionFactory();

        System.out.println(factory);
        System.out.println(factory.isClosed());
    }
}

```

```

Configuration cfg = new Configuration();
cfg.configure("hibernate.cfg.xml");
SessionFactory factory = cfg.buildSessionFactory();

```

```

//creating student
Student st=new Student();
st.setId(106);
st.setName("Durgesh Tiwari");
st.setCity("Lucknow");
System.out.println(st);
Session session= factory.openSession();
///

```

```

Transaction tx=session.beginTransaction();
session.save(st);
tx.commit();
session.close();

```

```

// Reading image
FileInputStream fis=new FileInputStream("src/main/java/pic.png");
byte[] data=new byte[fis.available()];
fis.read(data);
ad.setImage(data);
Session session= factory.openSession();
Transaction tx=session.beginTransaction();

```

```

session.save(st);
session.save(ad);

```

```

tx.commit();
session.close();
System.out.println("Done..");

```

Fetching Object

Session provided two methods for fetching data using Hibernate from database.

Get()

Use when we are not sure that the object is present or not.

It will return null if not present.

Load()

Use when we are sure the object is present.

It will return `ObjectNotFoundException` if not present.

Embeddable Object @Embeddable

It will tell to create the reference object fields with entity class.

For example if we have Student class and in this class we have id, name, address

Now address is another class. We have to annotate class as `@Embeddable`

Fetch Type in Hibernate.

Eager Loading - it will load data on query running.

Lazy Loading - it will load data only when we call `getter()` or `size()` method explicitly.

```
@OneToMany(mappedBy = "question", fetch = FetchType.EAGER)
private List<Answer> answers;
```

Hibernate Object Life-cycle States

- Transient State - when we create object.
- Persistent State - when we save this object means it will be associated with database and session.
- Detached State - It means we clear or close the session.
- Removed State - When we call delete. It means remove data from database but it is associated with session.

Hibernate Query Language HQL

```
String query="from Student where city='Lucknow'";
```

```
Query q=s.createQuery(query);
```

```
//single - (Unique)
```

```
//multiple-list
```

```
List<Student> list=q.list();
```

```
for(Student student:list)
```

```
{
```

```
    System.out.println(student.getName());
```

```
}
```

```

Transaction tx = s.beginTransaction();
Query q2 = s.createQuery("delete from Student s where s.city=:c");
q2.setParameter("c", "ABC");
int r = q2.executeUpdate();
System.out.println("Deleted:");
System.out.println(r);
tx.commit();

s.close();
factory.close();

```

Pagination in Hibernate

```

Query query = s.createQuery("from Student");

// implementing pagination using hibernate

query.setFirstResult(0);

query.setMaxResults(5);

List<Student> list = query.list();

for (Student st : list) {
    System.out.println(st.getName() + " : " + st.getCity());
}

s.close();

factory.close();

```

Cascading in Hibernate

It means it will store or save the data which is reference of other class implicitly.

We need to just add cascade attribute in this class field.

```

@OneToMany(mappedBy = "question", fetch = FetchType.LAZY, cascade = CascadeType.ALL)
private List<Answer> answers;

```

Caching in Hibernate

- Caching is the mechanism to enhance the performance of application.
- Caching is use to reduce the number of queries.
- Type of Caching

First Level – associated with Session Object – By Default

Second Level – associated with SessionFactory – Explicitly enable configuration

```
<!-- https://mvnrepository.com/artifact/net.sf.ehcache/ehcache -->
<dependency>
  <groupId>net.sf.ehcache</groupId>
  <artifactId>ehcache</artifactId>
  <version>2.10.6</version>
</dependency>
```

```
name="cache.use_second_level_cache">true</property>
name="cache.region.factory_class"> org.hibernate.cache.ehcache.internal.EhcacheRegionFactory</property>
```

```
@Entity
@Cacheable
@Cache(usage = CacheConcurrencyStrategy.READ_ONLY)
public class Student {
```

```
public class SecondCacheExample {
    public static void main(String[] args) {

        SessionFactory factory = new Configuration().configure().buildSessionFactory();

        Session session1 = factory.openSession();
        // first

        Student student1 = session1.get(Student.class, 12424);
        System.out.println(student1);

        session1.close();

        Session session2 = factory.openSession();
        // second
        Student student2 = session2.get(Student.class, 12424);
        System.out.println(student2);

        session2.close();
```

Hibernate Interview Questions

What is Hibernate and why to use?

Hibernate is ORM Object Relational mapping tool which map the Java object to database tables. It Provide us to JPA implementation to achieve this mapping with XML configuration or JPA Annotation.

Important Interface in Hibernate?

SessionFactory – org.hibernate.SessionFactory

It is an interface which is use to retrieve the session object.
It's like one SessionFactory per database connection.

Session – org.hibernate.Session

Its factory for Transaction. It use to get physical connection with database.
It provide the CRUD method operations.

Transaction - org.hibernate.Transaction

It will begin for CRUD operation process.

Q) What are hibernate configuration file

- contains database specific configurations and used to initialize SessionFactory.

- Conventionally, its name should be hibernate.cfg.xml

- If u need to connect to SQL then create

another one here

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 5.3//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-5.3.dtd">

<hibernate-configuration>

    <session-factory>
        <property name="hbm2ddl.auto">update</property>
        <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
        <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
        <property name="connection.username">codedecode</property>
        <property name="connection.password">codedecode</property>
        <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
        <mapping resource="employee.hbm.xml"/>
    </session-factory>

</hibernate-configuration>
```



Q) What are hibernate mapping file

The mapping file name conventionally, should be `class_name.hbm.xml`.

hibernate-mapping : root element

class : specifies the Persistent class.

id : specifies the primary key attribute in the class.

generator : used to generate the primary key.

property : specifies the property name of the Persistent class.

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 5.3//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-5.3.dtd">

<hibernate-mapping>
  <class name="com.codedecode.Employee" table="empTable">
    <id name="id">
      <generator class="assigned"></generator>
    </id>

    <property name="firstName"></property>
    <property name="lastName"></property>

  </class>
</hibernate-mapping>
```

Q) difference between openSession and getCurrentSession

getCurrentSession() method returns the session bound to the context.

Since this session object belongs to the context of Hibernate, it is okay if you don't close it. Once the SessionFactory is closed, this session object gets closed.

While

openSession() method helps in opening a new session.

You should close this session object once you are done with all the database operations. And also, you should open a new session for each request in a multi-threaded environment.

Q) difference between Session get() and load() method?

- get() loads the data as soon as it's called whereas load() returns a proxy object and loads data only when it's actually required, **so load() is better because it support lazy loading.**
- **Since load() throws exception** when data is not found, we should use it only when we know data exists.
- We should use **get() when we want to make sure data exists** in the database.

Q) hibernate caching – first level cache

- hibernate caches query data to make our application faster and improves performance.
- The idea behind cache is to reduce the number of database queries.
- Hibernate first level cache is associated with the Session object.
- Hibernate first level cache is enabled by default and there is no way to disable it.
- Still hibernate provides methods through which we can delete selected objects from the cache or clear the cache completely.
- Any object cached in a session will not be visible to other sessions and when the session is closed, all the cached objects will also be lost.

Q) configure Hibernate Second Level Cache using EHCACHE

- Add **hibernate-ehcache** dependency in your maven project, if it's not maven then add corresponding jars.

```
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-ehcache</artifactId>
</dependency>
```

- Add below properties in hibernate configuration file.

```
<property name="hibernate.cache.region.factory_class">org.hibernate.cache.ehcache.EhCacheRegionFactory</property>
<property name="hibernate.cache.use_second_level_cache">true</property>
<property name="hibernate.cache.use_query_cache">true</property>
<property name="net.sf.ehcache.configurationResourceName">/codecodeehcache.xml</property>
```

- Create EHCACHE configuration file, a sample file `codecodeehcache.xml` is as below:

```
<?xml version="1.0" encoding="UTF-8"?>
<ehcache xmlns:xsi="https://www.w3.org/2001/XMLSchema-instance"
    xsi:noNamespaceSchemaLocation="ehcache.xsd" updateCheck="true"
    monitoring="autodetect" dynamicConfig="true">

    <diskStore path="java.io.tmpdir/ehcache" />

    <defaultCache maxEntriesLocalHeap="10000" eternal="false"
        timeToIdleSeconds="120" timeToLiveSeconds="120" diskSpoolBufferSizeMB="30"
        maxEntriesLocalDisk="10000000" diskExpiryThreadIntervalSeconds="120"
        memoryStoreEvictionPolicy="LRU" statistics="true">
        <persistence strategy="localTempSwap" />
    </defaultCache>

    <cache name="employee" maxEntriesLocalHeap="10000" eternal="false"
        timeToIdleSeconds="5" timeToLiveSeconds="10">
        <persistence strategy="localTempSwap" />
    </cache>

    <cache name="org.hibernate.cache.internal.StandardQueryCache"
        maxEntriesLocalHeap="5" eternal="false" timeToLiveSeconds="120">
        <persistence strategy="localTempSwap" />
    </cache>

    <cache name="org.hibernate.cache.spi.UpdateTimestampsCache"
        maxEntriesLocalHeap="5000" eternal="true">
        <persistence strategy="localTempSwap" />
    </cache>
</ehcache>
```

- Annotate entity beans with @Cache annotation and caching strategy to use

```
import org.hibernate.annotations.Cache;
import org.hibernate.annotations.CacheConcurrencyStrategy;

@Entity
@Table(name = "ADDRESS")
@Cache(usage=CacheConcurrencyStrategy.READ_ONLY, region="employee")
public class Address {

}
```

How to Ingrate Hibernate with Spring?

We need to add dependency in pom.xml file. (Hibernate , JPA, Core Hibernate, MySql).

Create entity class, Controller class, Service class and DAO layer.